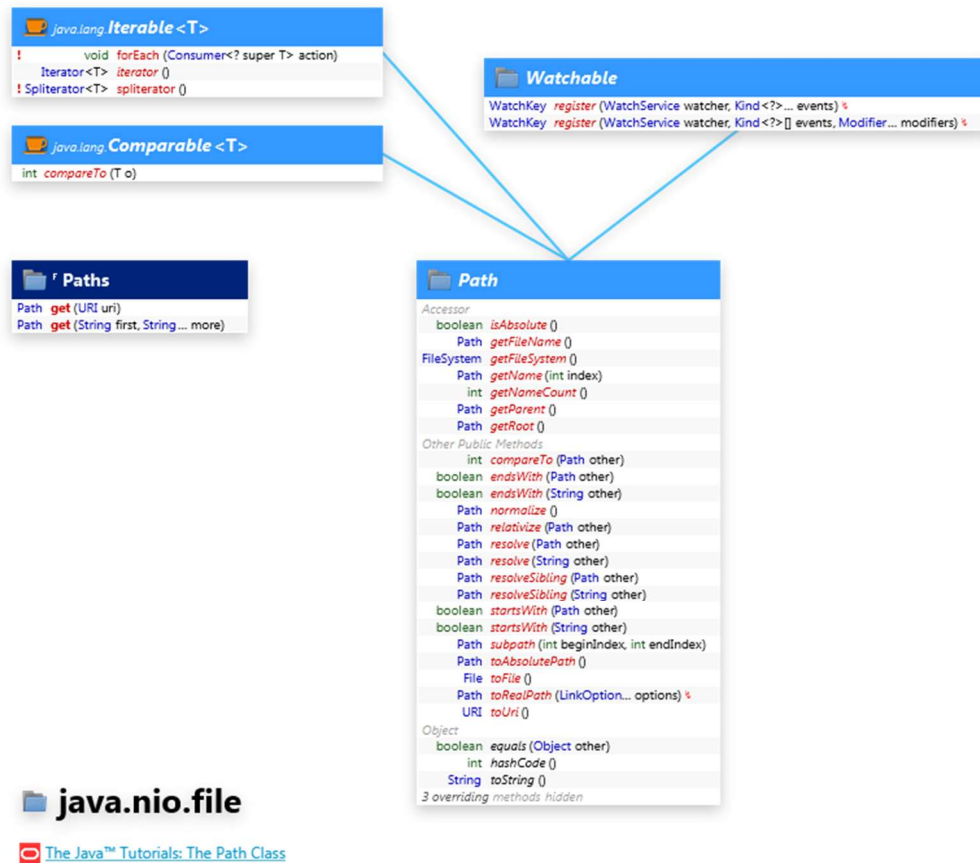


Java NIO.2 (java.nio.file)



Introducción: ¿Qué es NIO.2?

Java NIO.2 (desde Java 7) es la API moderna para trabajar con **sistema de ficheros, rutas, directorios, copias, borrados, permisos, walking, watch service**, etc.

Ventajas frente a java.io.File:

- API más completa y segura.
- Manejo correcto de excepciones.
- Soporte para enlaces simbólicos.
- Operaciones atómicas.
- WatchService para monitorizar cambios.
- Paths multiplataforma.

Bloques principales:

- **Path**
- **Files**

- **FileSystem**
- **FileVisitor**
- **WatchService**

2. Clase Path — El núcleo de NIO.2

Representa una **ruta** del sistema de ficheros.

Métodos esenciales

- **Path.of()** — crear rutas.
- **getFileName()**
- **getParent()**
- **toAbsolutePath()**
- **normalize()**
- **resolve()**
- **relativize()**

```
Path p = Path.of("/home/juanma/docs/archivo.txt");
System.out.println(p.getFileName());           // archivo.txt
System.out.println(p.getParent());              // /home/juanma/docs
System.out.println(p.toAbsolutePath());         // ruta absoluta
```

Clase Files — Operaciones de alto nivel

La clase **Files** es un *utility* con métodos estáticos para todo tipo de operaciones.

Métodos imprescindibles

- **exists()**
- **createFile()**
- **createDirectories()**
- **copy()**
- **move()**
- **delete()**
- **readAllLines()**
- **write()**
- **newBufferedReader()**
- **newBufferedWriter()**
- **walk()**
- **list()**

```
Path origen = Path.of("entrada.txt");
Path destino = Path.of("copia.txt");

Files.copy(origen, destino);
Files.move(destino, Path.of("mover/copia.txt"));
Files.delete(Path.of("mover/copia.txt"));
```

FileVisitor y Files.walkFileTree

Sirve para recorrer directorios de forma **profunda** y con control total.

Métodos clave del visitor

- **preVisitDirectory**
- **visitFile**
- **postVisitDirectory**
- **visitFileFailed**

```
Files.walkFileTree(Path.of("."), new SimpleFileVisitor<Path>() {
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes
attrs) {
        System.out.println("Archivo: " + file);
        return FileVisitResult.CONTINUE;
    }
});
```

WatchService — Monitorización de directorios

Permite detectar:

- Creación de archivos
- Modificación
- Borrado

Eventos principales

- **ENTRY_CREATE**
- **ENTRY_MODIFY**
- **ENTRY_DELETE**

```
WatchService ws = FileSystems.getDefault().newWatchService();
Path dir = Path.of("misdocs");

dir.register(ws, StandardWatchEventKinds.ENTRY_CREATE);

WatchKey key = ws.take();
for (WatchEvent<?> event : key.pollEvents()) {
    System.out.println("Evento: " + event.kind() + " -> " +
event.context());
}
key.reset();
```

Excepciones típicas en NIO.2

- **NoSuchFileException**
- **FileAlreadyExistsException**
- **AccessDeniedException**